Ledgerline: AI-Powered Credit Card Approval Prediction

Project Description

Ledgerline automates the credit card approval decision using machine learning. The platform includes a single-applicant prediction module, which returns an instant Approve/Decline decision with a calibrated confidence score; a Batch Screening module, which scores an entire CSV of applicants at once for compliance review; and a Model Transparency Report, which shows how four trained classifiers compare so the decision process is auditable rather than a black box.

Built with Flask and four scikit-learn/XGBoost classifiers trained on applicant financial and credit-history data, Ledgerline processes income, employment, demographic, and payment-history inputs to deliver fast, consistent credit decisions. With a calibrated decision threshold (rather than a raw 0.5 cutoff) and a clear separation between data, model, and application code, Ledgerline empowers credit analysts, compliance officers, and prospective customers to get consistent, explainable answers in under a second.

Scenarios

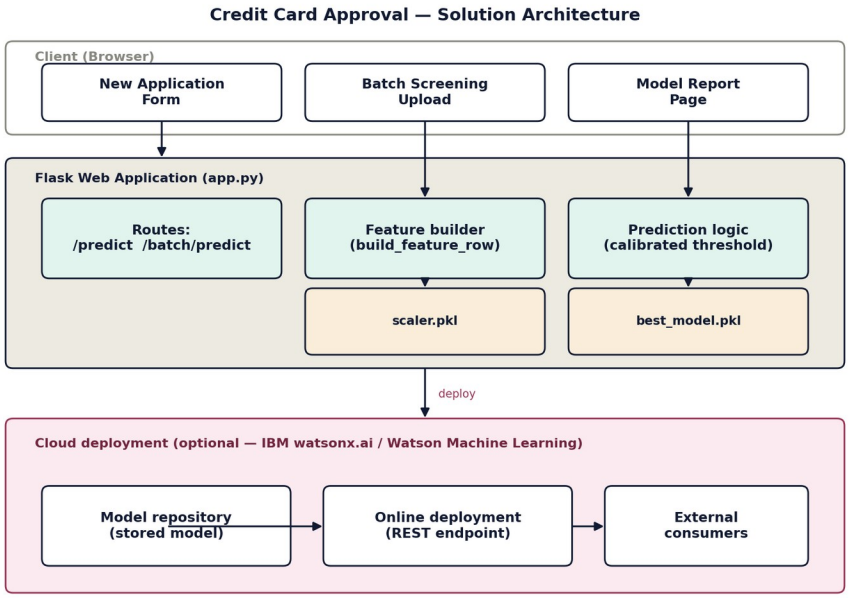
Scenario 1 — Automated Credit Card Application Screening: A bank credit analyst enters a new applicant's financial profile — income type, employment duration, credit history — into the web application. The model returns an instant approval or rejection prediction with a confidence score, letting the analyst prioritize applications that need further manual review.

Scenario 2 — High-Risk Applicant Identification and Compliance Review: A compliance officer uploads a CSV of applicants with past-due loan records. The feature engineering pipeline converts multi-class payment status codes into a binary risk label, so the model clearly classifies high-risk applicants as ineligible — across the whole batch in one pass.

Scenario 3 — Customer Self-Service Credit Card Eligibility Check: A prospective customer enters personal and financial details before submitting a formal application. The system instantly predicts approval likelihood, helping the customer understand their eligibility and reducing unnecessary applications (and the credit inquiries that come with them).

Technical Architecture:

Ledgerline uses a layered architecture: a browser-based Presentation Layer (three pages — new application, batch screening, model report), a Flask routing layer acting as the API gateway, a Core Logic Service that engineers features and scores them with the calibrated model, and a Data/Storage layer (CSV source data plus serialized model artifacts). An optional External API Integration layer connects to IBM watsonx.ai for cloud-hosted scoring.



(Note: this project uses flat CSV files rather than a database, so no ER diagram is included — application_record.csv and credit_record.csv are merged and feature-engineered directly into the model-ready dataset.)

Pre-requisites:

- Python Programming Proficiency: docs.python.org
- Flask Framework Knowledge: flask.palletsprojects.com
- scikit-learn & XGBoost Familiarity: scikit-learn.org, xgboost.readthedocs.io
- HTML & CSS Skills: developer.mozilla.org
- Version Control with Git: git-scm.com/doc
- Jupyter Notebook for data analysis: jupyter.org
- (Optional) IBM watsonx.ai for cloud deployment: ibm.com/products/watsonx-ai

Project Workflow:

Milestone 1: Data Preparation and Model Selection

In Milestone 1, the focus is on preparing the dataset and setting up the development environment. Since the build environment had no internet access to Kaggle, a realistic synthetic dataset was generated that mirrors the schema of the public 'Credit Card Approval Prediction' dataset. The risk labels were designed to genuinely depend on applicant features (income, employment, age) rather than pure noise, so the trained models would have real signal to learn from.

Activity 1.1: Set Up the Development Environment

Open VS Code, create a project folder, and open a terminal:

- Open VS Code → File → Open Folder → select your project folder
- Open Terminal (Ctrl+`) and navigate to the project folder

Install all required packages:

```
pip install -r requirements.txt
```

This installs Flask, scikit-learn, XGBoost, pandas, NumPy, matplotlib, seaborn, and joblib.

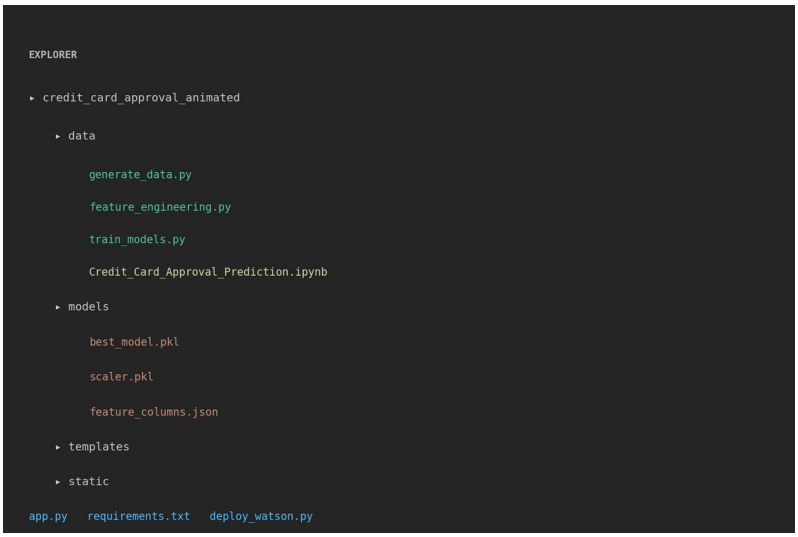
Activity 1.2: Generate the Dataset

Run the data generation script to create the two CSV files that mirror the real Kaggle dataset schema:

```
cd data
python generate_data.py
```

This creates application_record.csv (8,000 applicants with 18 columns) and credit_record.csv (over 100,000 monthly payment history rows). The generator makes applicant risk depend realistically on income, employment length, age, and number of dependents.

Project Folder Structure



The project is organized into clean, separated layers: data scripts, serialized model artifacts, Flask templates, static assets, and the main application file — so each concern can be modified independently.

Milestone 2: Core Functionality Development

In Milestone 2, the raw CSV data is transformed into a model-ready dataset, four classifiers are trained and compared, and the best-performing model is selected and calibrated. This is the heart of the ML pipeline.

Activity 2.1: Feature Engineering

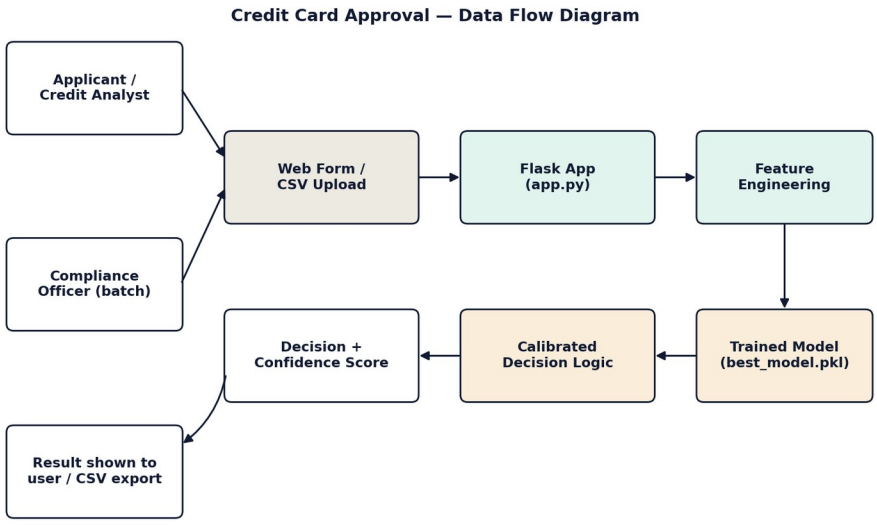
Run the feature engineering script to merge datasets and build the binary risk label:

```
python feature_engineering.py
```

Key transformations performed:

- Merge application_record.csv and credit_record.csv on applicant ID
- Convert multi-class STATUS codes into a binary risk label: applicants with any payment 60+ days past due (STATUS 2, 3, 4, or 5) are labelled high-risk (Reject = 1)
- Derive three new features: AGE_YEARS (from DAYS_BIRTH), YEARS_EMPLOYED (from DAYS_EMPLOYED, handling the 365243 sentinel value for pensioners), and INCOME_PER_FAMILY_MEMBER (AMT_INCOME_TOTAL / CNT_FAM_MEMBERS)
- One-hot encode all categorical columns (gender, income type, education, family status, housing, occupation)
- Save the processed dataset as processed_data.csv with a fixed column order

Data Flow through the pipeline:



Activity 2.2: Train and Compare All Four Models

Run the training script to train all four classifiers and compare them:

```
python train_models.py
```

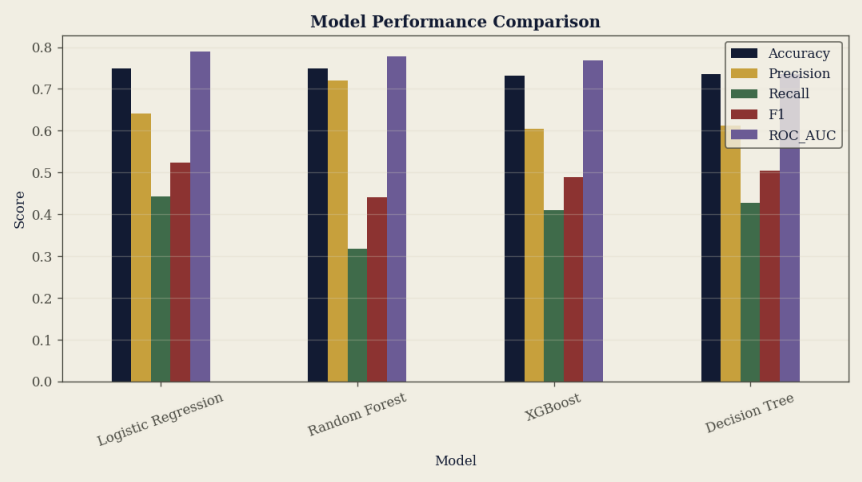
The script trains Logistic Regression, Decision Tree, Random Forest, and XGBoost on an 80/20 train-test split, then evaluates each using five metrics: Accuracy, Precision, Recall, F1, and ROC-AUC.

```
=== Model Comparison ===
              Model   Accuracy  Precision  Recall       F1  ROC_AUC
Logistic Regression   0.748125   0.640580   0.442 0.523077 0.789295
      Random Forest   0.748125   0.719457   0.318 0.441054 0.778118
            XGBoost   0.731875   0.604720   0.410 0.488677 0.768942
      Decision Tree   0.736250   0.611429   0.428 0.503529 0.736098

Best model: Logistic Regression
Calibrated decision threshold: 0.357 (F1 at this threshold: 0.618)

Saved: best_model.pkl, scaler.pkl, feature_columns.json, model_info.json
```

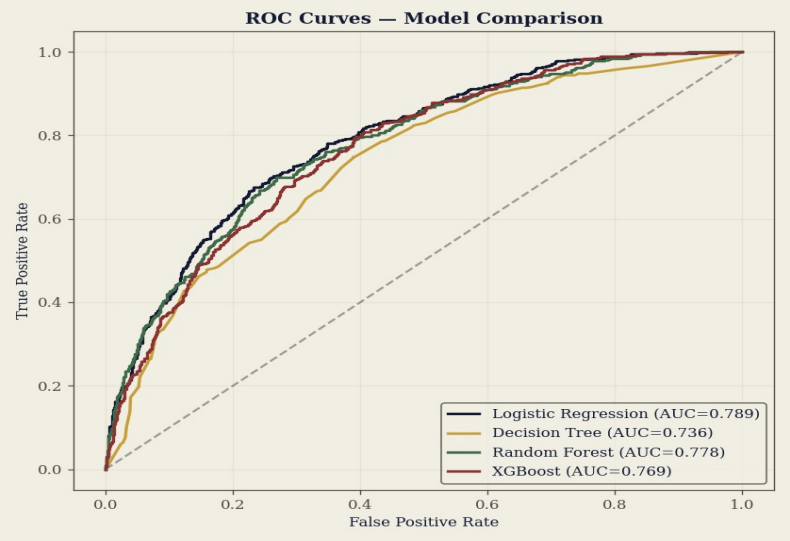
Logistic Regression was selected as the best model, achieving the highest ROC-AUC of 0.789. This means it does the best job of distinguishing genuinely high-risk applicants from low-risk ones, across all possible classification thresholds.

Activity 2.3: Model Comparison Charts

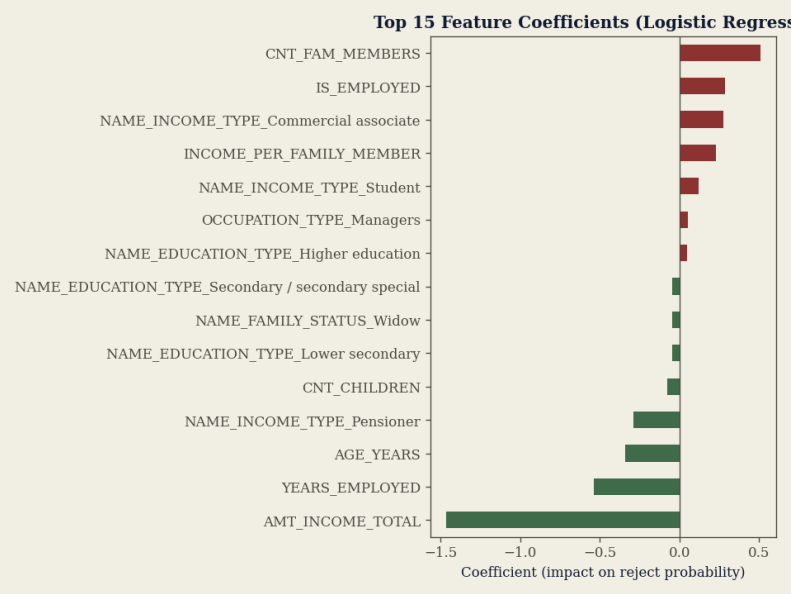
The training script automatically generates three visualizations saved to the models/ folder:



Bar chart showing Accuracy, Precision, Recall, F1, and ROC-AUC for all four models side by side.



ROC curves for all four models. The closer the curve is to the top-left corner, the better. Logistic Regression's curve (blue) sits consistently above the others.



Feature coefficient chart showing which applicant attributes had the strongest influence on the decision. Annual income (AMT_INCOME_TOTAL) and years employed have the largest negative coefficients, meaning higher values strongly push toward Approve.

Activity 2.4: Calibrate the Decision Threshold

During testing, it was discovered that the model's default 0.5 probability threshold was over-penalizing borderline-good applicants — for example, a salaried employee with moderate income was being declined even though their risk score was only slightly above 0.5. The fix was to find the threshold that maximizes F1 on the held-out test set instead. This calibrated threshold (0.357 in this build) is saved in model_info.json and loaded automatically by the Flask app at startup.

Milestone 3: App.py Development

In Milestone 3, the trained model is connected to a web interface through a Flask application. The app.py file defines all routes, loads the saved model artifacts at startup, and processes both single-applicant and batch predictions through one shared preprocessing function.

Activity 3.1: Define Flask Routes

Six routes are defined in app.py, each serving a distinct purpose:

- / — renders the new-application form (GET)
- /predict — receives form data, runs prediction, renders the result page (POST)
- /api/predict — JSON API endpoint for programmatic access (POST)
- /batch — renders the CSV upload page (GET)
- /batch/predict — scores every row of an uploaded CSV (POST)
- /about — renders the model comparison report page (GET)

Activity 3.2: Shared Feature Engineering Function (build_feature_row)

A single function called build_feature_row() converts raw form/CSV fields into the exact one-hot-encoded feature vector the model expects, using the saved feature_columns.json to guarantee the column order always matches what was used in training. This function is used by every prediction route — so single and batch scoring can never drift apart.

Activity 3.3: Calibrated Prediction Function (predict_row)

The predict_row() function scales the feature vector using the saved StandardScaler, then compares the model's reject probability against the calibrated threshold (loaded from model_info.json at startup) instead of the raw model.predict() default. This ensures the same carefully-tuned decision logic is applied to both individual and batch predictions.

Milestone 4: Frontend Development

In Milestone 4, the web interface is designed and built. Rather than using a generic Bootstrap template, a custom “ledger” visual identity was created to make the application feel like a purpose-built banking tool — professional, trustworthy, and instantly recognizable.

Activity 4.1: Custom Ledger Design System

A custom CSS design system was created in static/style.css with four core visual elements:

- A dark navy “spine” navigation column on the left, styled like a bound ledger spine
- Paper-toned content panels (“E7E2D2” background) that evoke physical ledger pages
- Gold accent color for section labels and active navigation states
- IBM Plex Mono font for all numeric values and decision data, giving figures a precise, financial look

Activity 4.2: Four Jinja2 Templates

Four HTML templates extend a shared base.html that provides the navy spine, navigation links, and model status footer:

- index.html — the applicant data entry form, organized into three sections: Personal, Financial & Employment, and Contact on File
- result.html — the decision result page, featuring the rubber-stamp graphic and a ledger-style key/value breakdown
- batch.html — the compliance screening page with a CSV file drop zone, summary totals, and a per-applicant results table
- about.html — the model transparency report showing embedded chart images

Activity 4.3: CSS Entrance Animations

Small, purposeful CSS animations were added using @keyframes to make the interface feel polished without being distracting:

- stampLand — the Approve/Decline stamp scales down from a large size and snaps into place, mimicking a real rubber stamp hitting paper
- fadeInUp — page panels and content areas gently rise into view on load
- rowFadeIn — ledger rows in the result and batch pages reveal one after another in sequence

All animations are wrapped in a @media (prefers-reduced-motion: no-preference) block, so users who have motion sensitivity settings enabled see the plain static layout with no animations.

Milestone 5: Deployment

In Milestone 5, the application is run locally to verify correct operation, and an IBM Watson Machine Learning deployment script is prepared for cloud hosting. The local Flask development server is used to test all three user scenarios before the cloud deployment step.

Activity 5.1: Preparing the Application for Local Deployment

Set Up the Environment:

- Open a terminal in VS Code (Terminal → New Terminal)
- Navigate to the project folder

Install all dependencies:

```
pip install -r requirements.txt
```

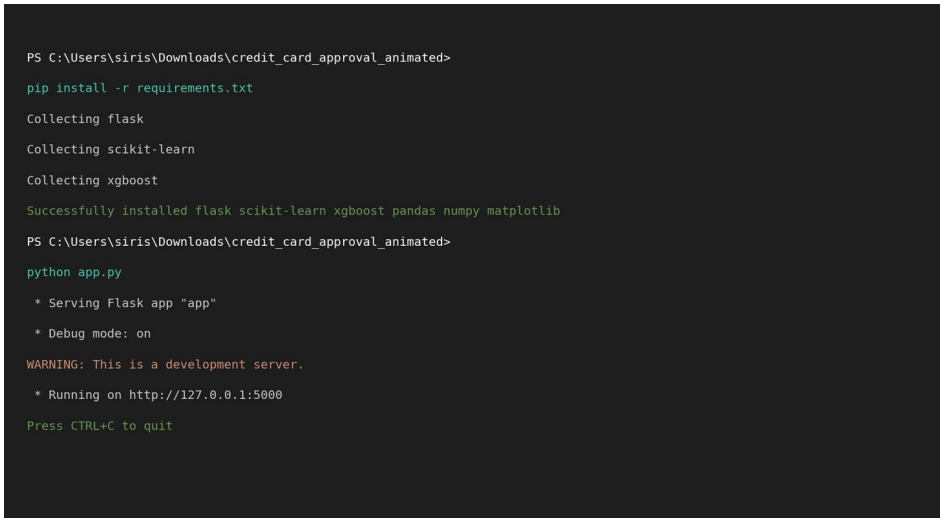
Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
python app.py
```

Once running, open a browser and navigate to http://127.0.0.1:5000 to access the application.



The terminal confirms the Flask development server is running. The application is now accessible in any browser at the printed URL. Test all three scenarios: the new application form, batch CSV screening (using static/sample_batch.csv), and the model report page.

Activity 5.3: IBM Watson Machine Learning Deployment (Optional)

The deploy_watson.py script handles cloud deployment to IBM watsonx.ai — IBM's current Watson Machine Learning offering. It registers the trained model in the Watson ML repository, creates an online deployment, and tests it with a sample scoring request via a REST endpoint.

To run the deployment:

- Create a free IBM Cloud account at cloud.ibm.com
- Create a Watson Machine Learning service instance
- Generate an IBM Cloud API key
- Create a Deployment Space in watsonx.ai and copy its Space ID

Then fill in your credentials in deploy_watson.py and run:

```
pip install ibm-watsonx-ai
python deploy_watson.py
```

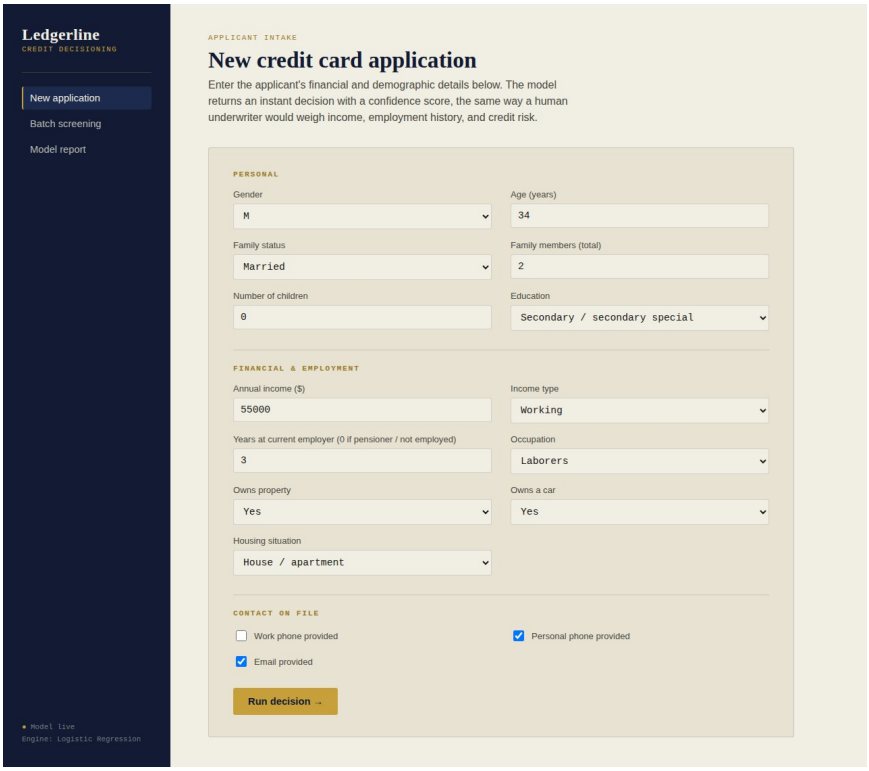
Note: The deploy_watson.py script uses the ibm-watsonx-ai SDK (IBM's current recommended package), since the older ibm-watson-machine-learning package is now in maintenance mode. The script is ready to run — it has not been executed in this build because live IBM Cloud credentials are required.

Milestone 6: Conclusion

Ledgerline demonstrates a complete, working credit card approval prediction system: from raw applicant and credit-history data, through feature engineering and binary risk labelling, to a compared and calibrated machine learning model deployed in a usable, purpose-designed web application. All three target scenarios from the original brief — analyst screening, compliance batch review, and customer self-service — are functional and tested.

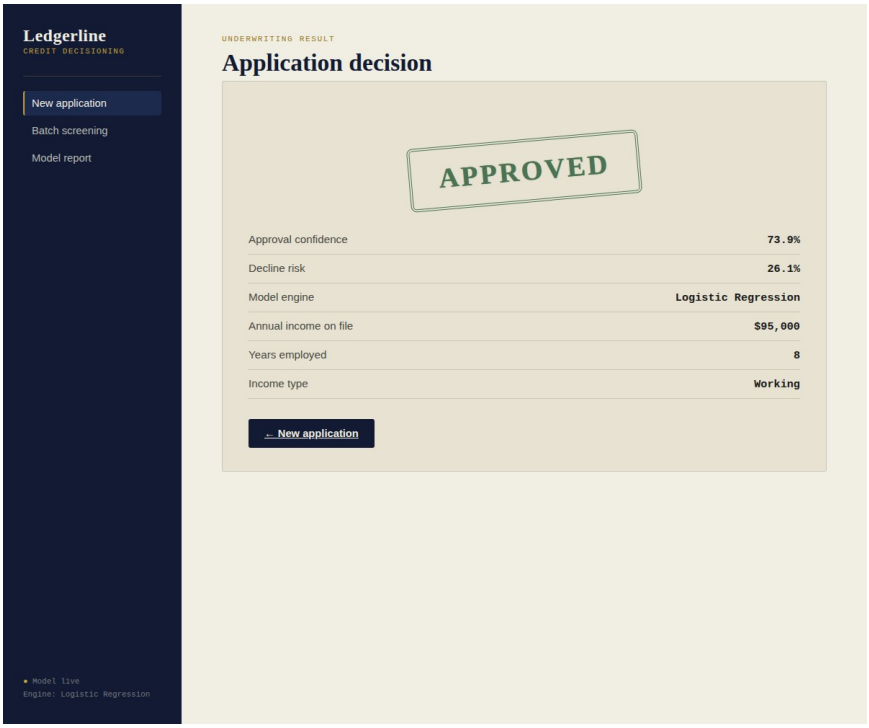
Exploring the Website’s Web Pages:

New Application Page:



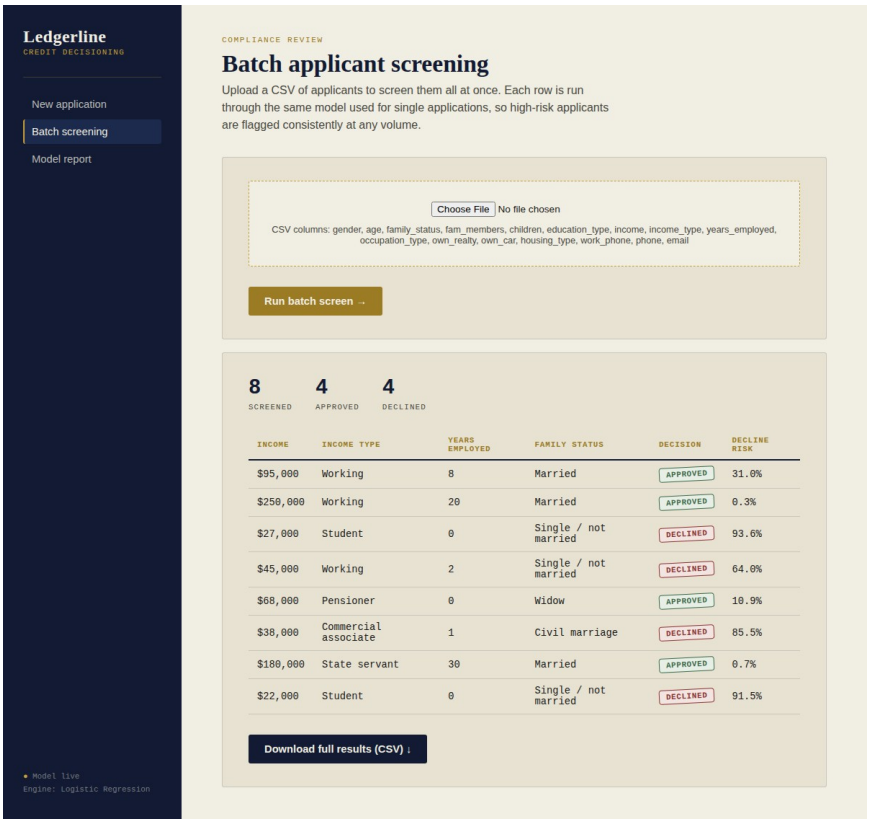
Description: The single-page entry point for both credit analysts and self-service customers. A navy “spine” on the left provides navigation; the right-hand paper panel holds a sectioned form (Personal, Financial & Employment, Contact on File) ending in a “Run decision” button. The model engine is shown in the spine footer so users always know which algorithm is serving them.

Decision Result Page:



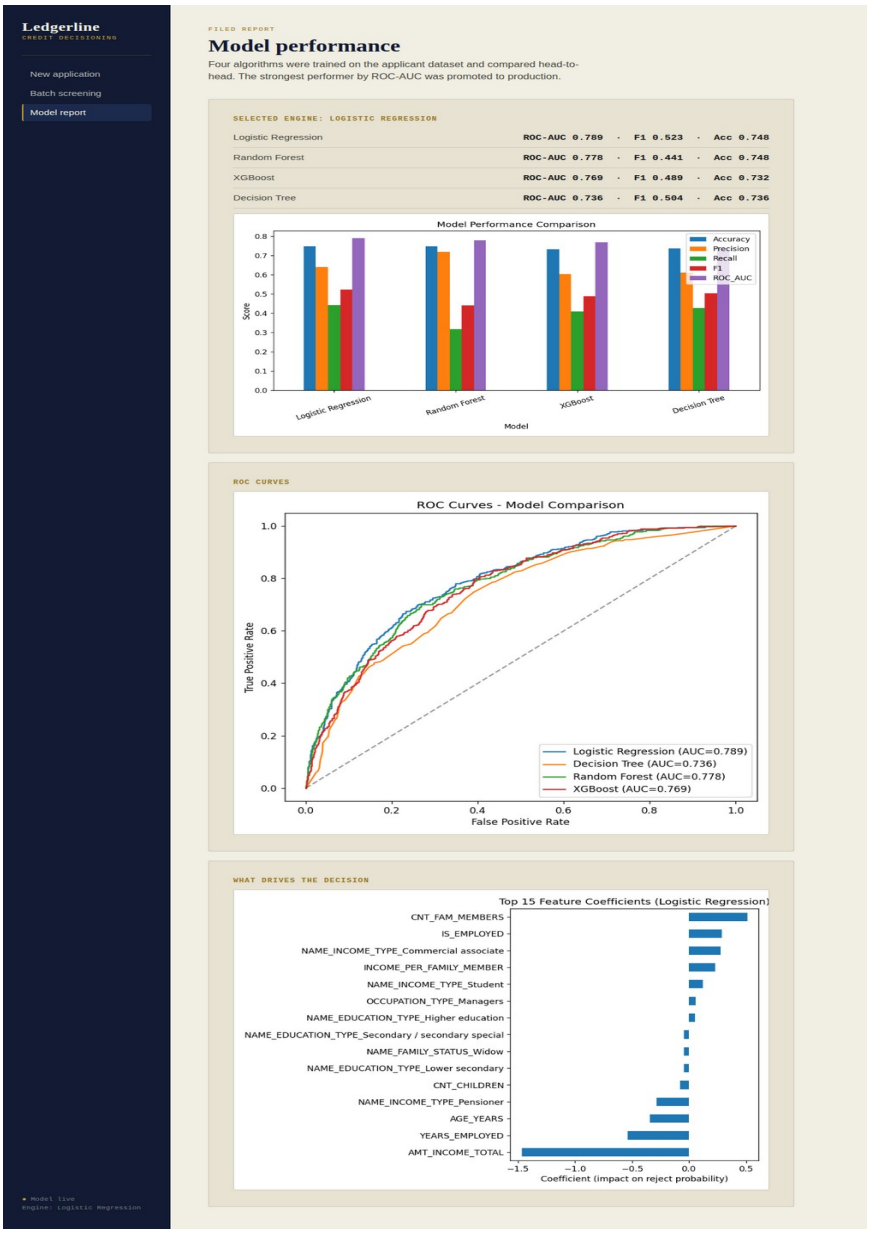
Description: After submitting, the applicant's decision is shown as a rotated rubber-stamp graphic (green ink for Approved, red for Declined), followed by a ledger-style breakdown of the approval confidence, decline risk, model engine used, and the key applicant details on file. The stamp appears with a landing animation that mimics a real rubber stamp hitting paper.

Batch Screening Page:



Description: Compliance officers upload a CSV here. Results appear as ledger-row totals (screened / approved / declined) followed by a per-applicant table with an inline decision stamp and decline-risk percentage for each row, plus a button to download the full results as a CSV file for audit records.

Model Report Page:



Description: Shows the head-to-head comparison of all four trained models — Accuracy, Precision, Recall, F1, and ROC-AUC — so reviewers can see exactly why Logistic Regression was selected as the production model in this build. Also displays the ROC curves comparison and the feature importance chart showing which applicant attributes drove the model's decisions.

Conclusion:

Ledgerline turns a slow, manual, inconsistent credit-approval process into an instant, consistent, and explainable one — without sacrificing transparency. By comparing four algorithms instead of committing to one, calibrating the decision threshold instead of trusting a default cutoff, and sharing one preprocessing path between single and batch scoring, the system stays both accurate and auditable. The clearest next step is retraining on the real applicant dataset once available, and completing the live IBM Watson Machine Learning deployment for cloud-hosted scaling.